

SimplArchive

User Manual

An enterprise-grade document management system
— the web workbench and the desktop client

© 2026
Hebel Consulting GmbH
Schweighofplatz 7
6010 Kriens (LU)
Switzerland
support@simplarchive.dev

Contents

1	Introduction	3
2	Getting started	4
3	Browsing & previewing documents	5
4	Adding & versioning documents	6
5	Organizing	11
6	Working from your file manager (WebDAV)	12
7	Contacts, calendars & your tasks	15
8	Your own device	27
9	Metadata & classification	29
10	Search	30
11	Collaboration	31
12	Sharing outside SimplArchive	33
13	Workflow & records	36
14	Administration & account	37
15	Appendix — glossary & links	39
16	Appendix: the desktop client's log	40
17	Appendix — who does what	41
18	Appendix — how SimplArchive is put together	43
19	Appendix — security posture	46
20	Index	50

1 Introduction

SimplArchive is a multi-tenant **document management system** (DMS): a secure archive where an organization files, versions, classifies, searches, and governs its documents. It is a showcase of how a senior, AI-driven software developer can produce a complex, enterprise-grade system in a relatively short period — every feature in this manual is really implemented and really tested.

You reach the same archive through **two clients**, which mirror each other feature for feature:

- the **web workbench** — a browser application, nothing to install;
- the **desktop client** — a native Windows / macOS / Linux application that can additionally open a document in its real desktop program and drag documents in and out of the operating-system file manager.



Figure 1: The two clients: the web sign-in (left) and the desktop logon window (right).

1.1 Core concepts

Repository — a top-level archive; it is simply a folder with no parent. **Folder / document** — the tree inside a repository. **Version** — every change to a document's file is kept as a new version; nothing is overwritten. **Mask (document type)** — a template of **index fields** (metadata) attached to a document. **ACL** — a per-document access-control list of **rights** (see, read, edit, ...) granted to users and groups.

2 Getting started

Signing in. Open the web client and choose **Log in**, or start the desktop client and use its logon window; enter your e-mail and password. Your organisation may additionally require a second factor (a one-time code or a passkey). You can pick the interface **language** (English, German, Italian, Spanish) and switch between a **light** and **dark** appearance at any time.

If sign-in suddenly refuses everything, you have most likely mistyped several times in a row. Repeated failed attempts are turned away for a short while — about a minute at first, longer if they keep coming — and the wait clears on its own, so there is nobody to ring and nothing to reset. A passkey keeps working throughout, which is the quickest way back in if you have one.

The workbench. After signing in you land on the **Repositories** workbench, laid out as: the **tree** of repositories and folders, the **contents** list of the selected folder, the **detail** pane (index data) over the **preview**, and — along the bottom — the **tab bar** that switches between Repositories, Intray, Search, Tasks and the rest.

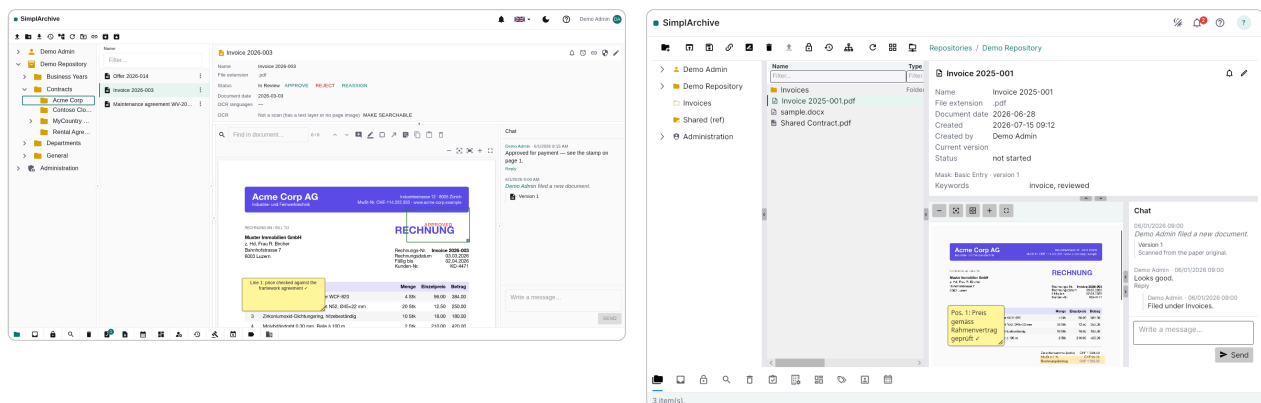


Figure 2: The workbench in the web client (left) and the desktop client (right): tree · contents · detail · preview, with the bottom tab bar.

3 Browsing & previewing documents

Expand the **tree** to a folder and its documents appear in the **contents** list, which you can sort by any column. Selecting a document shows its **index data** in the detail pane and renders a **preview** below — PDFs, images, and converted Office/e-mail/Markdown documents alike. Full-text search hits are highlighted directly on the preview, and you can click any word to copy it — **shift-click** appends instead of replacing, so a few clicks collect a phrase (an invoice number, then its date) without touching the keyboard. That is the fast way to fill index fields on the **Intray** tab: click the values in the scan's preview, paste them into the field. In the desktop client you can also **open** the document in its native application — from the row's context menu, the ribbon, a double-click, or the keyboard shortcut **⌘O** (**Ctrl+O** on Windows and Linux). The same shortcut opens the selected item on the **Intray** tab.

The preview's toolbar carries the **zoom** controls. A document opens fitted to the width of the pane; *fit page* shrinks it until the whole page is visible at once — useful when the pane is wider than it is tall, where fitting the width pushes the bottom of the page out of sight. Zooming out after that walks back down to the whole page and stops there. **Ctrl+scroll** (**⌘+scroll** on a Mac) zooms as well.



Figure 3: The preview with search hits highlighted on the page — click a word to copy it (shift-click to append), or step through the matches.

4 Adding & versioning documents

4.1 How documents get in

There is more than one way in, and which you want depends on whether the document is finished, whether it belongs to something already filed, and how many you have. They all end in the same place.

Route	Use it when
Upload on the ribbon	You are already looking at the folder it belongs in. Picks files and files them straight into the open folder.
Drag onto a folder — a row in the list, or a node in the tree	You can see the destination. The document is filed there; the view follows to that folder so you can see it arrive.
Drag onto empty space in the contents list	Same as above for the folder you are already in.
Drag onto a document	The file is a newer copy of that document — it is added as a version , not as a second document, and the history is kept.
Drag onto your own space ► Inray	It is not ready to file, or you do not yet know where it belongs. It waits in the Inray until you classify and file it.
Drag a document onto your own space ► Inray	You want to start from an existing document as a template . A copy lands in your Inray carrying that document's document type and index data, so you edit what differs. Nothing is created in the archive until you file it.
Drag onto your own space ► Check-out	You checked a document out, edited it on your computer, and are bringing it back. The file must still carry the document's name — that is what says which document it belongs to.
WebDAV	You would rather work in Finder, Explorer or Files. Mount the archive as a drive and copy documents in like any other folder.
Import	You are bringing in a whole folder tree at once, exported from SimplArchive or elsewhere.
Email attachment	You filed an email and want one of its attachments as a document of its own.



Figure 4: Your own space expanded — it carries **your name** at the top of the tree: **Intray** and **Check-out** sit above your own folders. Drop files on **Intray** to stage them, or drop an edited working copy on **Check-out** to bring it back — and drag a document onto **Intray** to start new work from it as a template.

Two of these do not create a document. A drop onto the **Intray** stages an item — it becomes a document only when you file it. A drop onto **Check-out** replaces your working copy — the document gets a new version only when you check it in. Everything else files immediately.

Uploading. Drag a file straight onto a folder (the bytes go directly to object storage — the server never proxies them). The **Intray** is a staging area: drop scans or files there, then classify each one (name, document type, index data) and **file** it into the archive.

What the archive will not keep. Programs and scripts are refused — an .exe, a .bat, a shell script, and anything that turns out to be a program whatever it has been named. Everything else is kept, including the formats a document system is often too narrow for: drawings, disk images, archives, video. When an **email's** attachment is refused the message itself is still archived, and a line in its chat thread names the attachment that was left out, so nothing goes missing quietly.

When the name is already taken. A folder cannot hold two things of the same name, so filing Invoice.pdf where an Invoice already sits asks you what you meant rather than refusing:

Choice	What happens
A new version of that document	The usual answer: the file is a newer revision of what is already there. The document keeps its index data and its history, and the previous version stays available.
A new document with a different name	The two are genuinely different documents that happen to share a file name. A free name is offered — Invoice (2) — and you can change it.

Either way you can add a **comment** saying what this file is, which becomes the version's comment. There is no “overwrite”: nothing in SimpliArchive replaces content in place, and the honest equivalent is a new version.

If the name is held by a **folder** rather than a document, only the second choice is possible — a folder has no versions to add one to.

Versions. Uploading a new file to an existing document adds a **version** — the history is preserved. The **Versions** dialog lists every version; you can compare two of them or **make current** an older one. When you upload a file that already exists, SimplArchive warns you of the **duplicate**.

Give each version a **comment** saying what changed. It is the difference between a history someone can read and a list of timestamps: “Price corrected after the framework-agreement review” answers the question a reader of the list actually has, and no date can.



Figure 5: The **Versions** dialog: every version of the document with who saved it, when, and — the part that makes the list worth reading — the comment describing what changed.

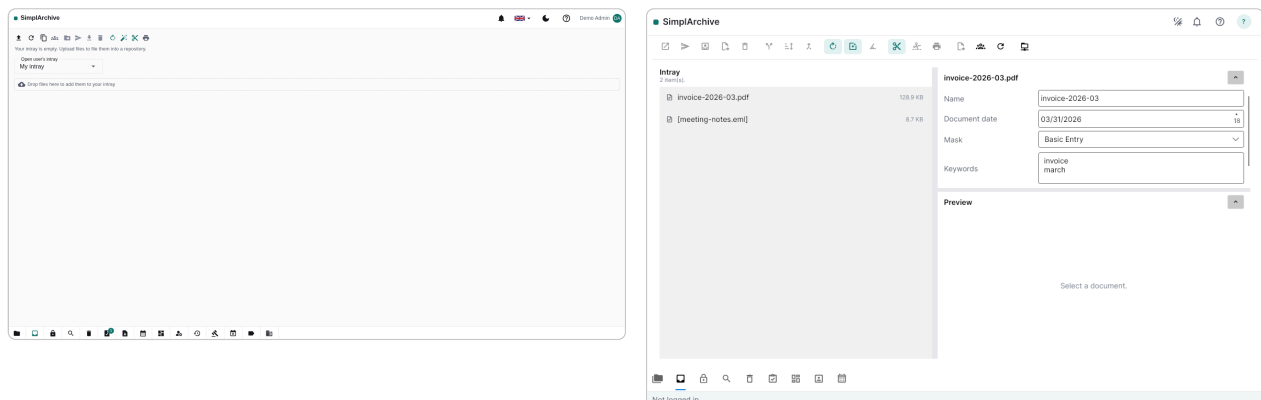


Figure 6: The Inray: staged items waiting to be classified and filed, in the web (left) and desktop (right) clients.

4.2 Tidying a scan before filing

A scan rarely arrives filing-ready: a batch holds several documents, a page went through the feeder sideways, a blank back rode along. The Inray is where all of that is put right — **before** filing, because a staged item can be reshaped freely, while a filed document's pages are part of the record. Every operation here works on .pdf and multi-page .tif files; anything else simply does not offer them.

Operation	What it does
Split into pages	One new item per page. The original is kept, so a split that turns out wrong is undone by deleting its output — a scan can be the only copy of a piece of paper.
Rotate/Sort	Opens the page dialog below: put the pages in order, delete a page with the bin on its tile, and turn a page with the   buttons under it. Nothing is written until Apply order — one save for the whole arrangement, and Cancel discards everything. Offered from a single page up, because rotating a one-page scan that went in upside-down is exactly what it is for.
Join items	Several staged items become one, in the order you chose them. The sources are kept.
Cut at separator sheets	Cuts a batch into one item per document at the printed separator sheets between them (see below).



Figure 7: The **Rotate/Sort** dialog on the sample batch: page 4 went through the scanner upside-down and has been turned a quarter so far — one more press of its  button puts it upright. The bin on each tile deletes that page; nothing is saved until **Apply order**.

Rotation does not damage a PDF. Turning a PDF page only records the new orientation — the page’s real, searchable text is untouched. A .tif page has no such notion and is re-encoded when turned, the same trade its straightening makes.

Separator sheets. For a stack of paper documents, print the **separator sheet** (the printer button in the Intray toolbar), lay one between each document, and feed the whole pile through the scanner in one go. **Cut at separator sheets** then produces one item per document and discards the sheets themselves. Sample files to try this with — a batch as the scanner would produce it, and the sheet — are served by your own installation under /download/samples/.

The automatic toggles. Three sticky switches in the Intray toolbar act on every **arriving** upload — including files saved into the Intray folder of the mounted network drive:

Toggle	What it does to an arriving scan
Auto-rotate	Turns pages that arrived upside-down or sideways the right way round. Works for .pdf and .tif.
Auto-straighten	Corrects the slight skew of a crooked scan — for .tif, and for a PDF that is itself a pure scan (image pages, no real text). A born-digital PDF is left alone: straightening re-renders the pages, and doing that would replace its real text with a recognised approximation.
Cut at separator sheets	Applies the separator cut to every arriving batch, so a scanner that feeds straight into the Intray needs no manual step at all.

Digitally signed documents are never touched. Every page operation would break the signature, so on a signed item none is offered and the automatic steps skip it — the refusal is the feature.

Already filed? Rotate/Sort also exists on the **Check-out** tab. It edits the **working copy** of a document you have checked out — the archived version stays exactly as it was, and your rearrangement becomes the new version only when you press **Check in** (or is thrown away by **Discard**, like any other working-copy edit). Split and join stay in the Inray: they turn one item into several documents, which is filing work.



Figure 8: **Compare versions:** two revisions of a document side by side — old on the left, new on the right, changed lines aligned and the changed words highlighted within them. Works for any pair whose content yields text, notes and emails included.

5 Organizing

Create **folders**, and **move** documents between them by drag-and-drop. A **reference** (shortcut) lets one document appear in several places without copying it. **Tags** label documents for quick grouping — your tenant can maintain a curated tag catalogue with colours. **Sensitivity labels** mark how confidential a document is. Every user also has a **personal repository** for private documents.



Figure 9: The tag catalogue: curated, colour-coded tags an administrator maintains for the whole tenant.

6 Working from your file manager (WebDAV)

The whole archive is reachable over **WebDAV** as a network drive — not just your personal space but the shared repositories you have permission to access, with your rights enforced on every operation. The mounted drive is called **SimplArchive** and mirrors the Repositories tree exactly: your Personal space, then the repositories you can see, and nothing else.

The **WebDAV** button does the next useful thing rather than always the same thing, and its tooltip says which. In the **desktop client**: if you have no credentials yet it opens the setup dialog; if you have credentials it mounts the drive and opens it (on Windows as a persistent drive letter, S: or the next one free); and if the drive is already mounted it goes straight there. So “show me my documents in Finder” is one click once you are set up.

It opens where you already are. There is one button per tab, and each opens **its own** folder inside the drive rather than the top of the archive:

Where you press it	What opens
Repositories ribbon	The folder selected in the tree — the drive mirrors the tree exactly, so “where I am” and “which folder on the drive” are the same place. With nothing selected, the whole archive opens.
Intray tab, lower left	Your Intray folder.
Check-out tab, lower left	Your Check-out folder, where working copies live.

If you connect to more than one SimplArchive — a local one and a hosted one, say — your computer cannot give both drives the same name, so the second is called something like **SimplArchive-1**. The button always opens the drive belonging to the server you are signed in to, whichever name it ended up with.

The setup dialog shows the **mount URL** and your **username** (your e-mail), each with a **Copy** button, and a **Generate** button that issues an app-specific **WebDAV password** — separate from your login password and shown only once, so copy it right away. The desktop dialog also has **Open folder**, so you can go from generating credentials to a mounted drive without closing it.

The **web client** cannot mount a drive: a browser is not allowed to. Instead of leaving you with a URL and no idea what to do with it, its dialog shows the mount steps for the operating system you are on — Finder’s **Go** ► *Connect to Server* on macOS, **Map network drive** on Windows, a `davs://` address on Linux — next to the values to paste into them.



Figure 10: The web client’s WebDAV dialog: the mount URL, your username and the one-shot password, each with a copy button, and the mount steps for the operating system you are using.

6.1 Editing a document in place

Open a document from the mounted drive, change it, save it — that is the whole gesture, and it works the way you would expect from any network drive.

What happens underneath is worth knowing, because it changes what other people see. Simple editors write the file straight back, and the change becomes a new version immediately. An **office suite**, however, never overwrites a file in place: it writes a hidden companion file, then a temporary copy of the new content, and finally renames that copy over the original. SimpliArchive recognises that sequence and turns it into a **check-out** — the same one you get from pressing **Check out** in the app.

So after saving from your file manager:

- the document is **checked out to you**, and appears on your **Check-out** tab marked **automatic**;
- your edit is waiting there as your working copy — it is **not yet a new version**, and other people still see the previous one;
- nobody else can change the document until you are finished;
- you finish by pressing **Check in**, from either client or by saving into the mount’s **Check-out** folder.

Saving is not the same as publishing. The system cannot tell when you have finished editing — only you can, and **Check in** is how you say so. Until then your work is kept safely as your working copy. A check-out left untouched for a long time is released automatically by your organisation’s idle rule, and you are warned before that happens.

The **automatic** marker exists so this never looks like a fault: a document you never explicitly checked out has become yours, and the tab says why, naming the program that did it.

If someone else already has the document checked out, your editor is told the file is locked and opens it read-only, rather than letting you edit for ten minutes and failing at the save. Documents you are not allowed to edit, and documents frozen by a legal hold, behave in the mount exactly as they do in the app: they refuse.

A warning from your own software is not a fault in ours. Some commercial vendors treat a WebDAV connection to a site that is not on their own allow-list as suspicious, or refuse it outright. That is a policy

decision in the respective vendors' software, not a limitation of SimplArchive — the same mount that one program declines will typically work in another on the same machine.

6.2 Reading the archive in your mail program (IMAP)

Where WebDAV turns the archive into a network drive, **IMAP** lets a mail program browse it: point any IMAP-speaking mail client at your SimplArchive server and browse the same ACL-filtered tree as mailboxes. **INBOX** is your personal folder; the shared repositories you can see appear beside it. Archived e-mails read natively; every **other** document can appear too — as a message carrying the file as an attachment — if you switch that on.

This is your archive wearing an IMAP face — it is not a mail service. SimplArchive does not collect your mail from your provider, does not send mail, and is not a replacement for your e-mail account. Nothing here changes where your mail lives. What it gives you is the second account in your mail program: keep your real one alongside it, and **file** mail across from one to the other, which is what the drag described below is for.

Shortcuts show up as mailboxes too. A folder you have filed a reference to appears in the list wherever the reference sits, with everything beneath it — so the destinations you file into most can be gathered in your personal folder and reached from the mail program without hunting through the whole tree.

Open **Email access (IMAP)**... in the account menu. The dialog shows the **server** and your **username** (your e-mail), each with a **Copy** button, and **Generate** issues a dedicated **IMAP password** — like the WebDAV one: separate from your login password, shown only once, revocable on its own. The **Show every document** switch is yours per account: off (the default) lists only e-mails; on lists everything you may see, with the file attached to open straight from the mail client.

The mail client cannot rearrange the folders. Folders are created, renamed and deleted in SimplArchive — an IMAP client trying the same is politely refused. Messages, though, are yours to work with: drag an e-mail into a folder to **file it into the archive** (it becomes a proper e-mail document, named by its subject), move a message to re-file it, copy one to leave a **reference**, and delete + expunge to send it to the **recycle bin** — never a hard delete. Read/unread marks are remembered per person.

Notes, too. Point a notes app that syncs over IMAP at the same account and it finds the **Notes** mailbox — your personal **Notes** folder in disguise. Every note becomes a proper archive document with the **Note** document type, and editing a note on your phone adds a **new version** in the archive rather than a new copy — the full history stays browsable in the workbench. The Notes folder is typed: it accepts only notes, and notes live only there (references to them may go anywhere).

7 Contacts, calendars & your tasks

Two of the workbench tabs are not about documents at all — or rather, they are about documents that happen to be a **contact card** or a **calendar entry**. **Contacts** lists the people in your addressbooks; **Calendar** lists your appointments. Both read the same archive, with the same permissions, and both can be **subscribed to from your phone**.



Figure 11: The Contacts tab: addressbooks on the left, the people in the ticked ones merged into one list.

Collections overlay rather than replace. The left-hand list is a set of tick-boxes, not a single choice — tick two addressbooks and you see both sets of people in one list, each row carrying a small colour swatch saying which collection it came from. Your own **My Addressbook** and **My Calendar** start ticked, so the tab opens with something in it. The colour is yours to set and yours alone: changing it does not change what anybody else sees.

Every user's personal space contains **My Addressbook** and **My Calendar** from the start. They cannot be deleted or moved — they are the fixed points your phone subscribes to, and provisioning them would be pointless if the next click could remove them. Addressbooks and calendars can also live **anywhere else** in the archive: a shared repository can hold a team addressbook, and whoever has permission sees it in the same list.

7.1 Adding and editing a contact or an appointment

New contact and **New appointment** open a form. Fill it in and save, and the item is created — nothing exists until you save, so closing the form leaves nothing behind. If more than one collection is ticked, the form asks which one it should go into; with only one candidate it does not ask, and the confirmation names where it went.

Edit opens the **same** form on an existing item. That is deliberate rather than incidental: a create form with fewer fields than the editor would quietly discard whatever you typed into the ones it lacked.

A contact holds rather more than a name — several **e-mail addresses** and **phone numbers**, each with its own type, several **postal addresses**, an organisation, a job title, a birthday, a website and a note. Add and remove rows as you need them.

Everything the form does not show is kept. A contact created on a phone carries things this form does not model — a photo, custom labels, fields belonging to some other program. Saving here **preserves** all of it rather than dropping what it does not understand, which is what makes it safe to edit in the archive a card that lives on your phone.

7.2 Advanced: the stored item

Under every contact and appointment form there is a collapsed **Advanced: the stored item** section. Open it and you see the item exactly as stored — a **vCard** for a contact, an **iCalendar** entry for an appointment — including the properties the form above does not show.

The screenshot shows a contact editor form. The top section contains form fields for personal and contact information: First name (Anna), Last name (Meyer), Organization (Contoso), Job title (Head of Procurement), Email (work: anna@example.test, home: anna.private@example.test), Phone (mobile: +41 79 000 00 00), Address (work: Bahnhofstrasse 1, Zurich, 8001, Switzerland), Birthday (1990-02-15), Website (https://contoso.example), and Note (Met at the trade fair.). Below these fields is a collapsed section titled "Advanced: the stored item". When expanded, it shows a text box containing the raw vCard data:

```
BEGIN:VCARD
VERSION:3.0
UID:6f1c2e40-1
FN:Anna Meyer
ORG:Contoso
TEL;type=WORK:+41 79 000 00 00
X-ABShowAs:COMPANY
PHOTO;ENCODING=b:/9j/4AAQSkZJRg...
END:VCARD
```

At the bottom of the form are "Cancel" and "Save" buttons.

Figure 12: The contact editor with **Advanced: the stored item** open — the stored vCard, including the properties no form field shows.

It is editable, and this is the one place in the product where saving **replaces** rather than merges. Delete a line here and the property is gone; that is what “raw” has to mean, and a merge would put it back while telling you the save had succeeded. Two things are refused rather than accepted: text that is not a valid card or entry, and a change to the **UID** — the identifier every phone and mail program uses to recognise this item. Changing the UID would not rename the item, it would make it a **different** one, so your phone would keep the old copy and add the new. In both cases nothing is written and the stored item is untouched. Removing the UID line is not a change; the stored one is kept.

While you are editing the raw text, the fields above go read-only. Both describe the same item and only one of them can be saved. A line under the box says which way the save will go.

Every save — from the form or from the raw box — writes a **new version**, exactly as any other change to a document does. A raw edit you regret is recoverable from the version history like anything else.

7.3 Subscribing from your phone (CalDAV & CardDAV)

Addressbooks and calendars speak **CardDAV** and **CalDAV** — the standards phones, tablets and mail programs already use. Point a device at your SimplArchive server and it finds every addressbook and calendar you are allowed to see, wherever it sits in the archive; you then pick which to sync. Edits travel both ways: a contact changed on your phone becomes a new version in the archive, and one changed in the workbench appears on the phone.

The credential is the **same DAV password** the file-manager mount uses — one secret, one place to revoke it. Issue it from **WebDAV...** in the account menu if you have not already.



Figure 13: The Calendar tab, ordered by time — undated entries sort last rather than into the top rows.

7.4 Your tasks, on your phone

Beside your calendars, two more collections appear that hold no documents at all:

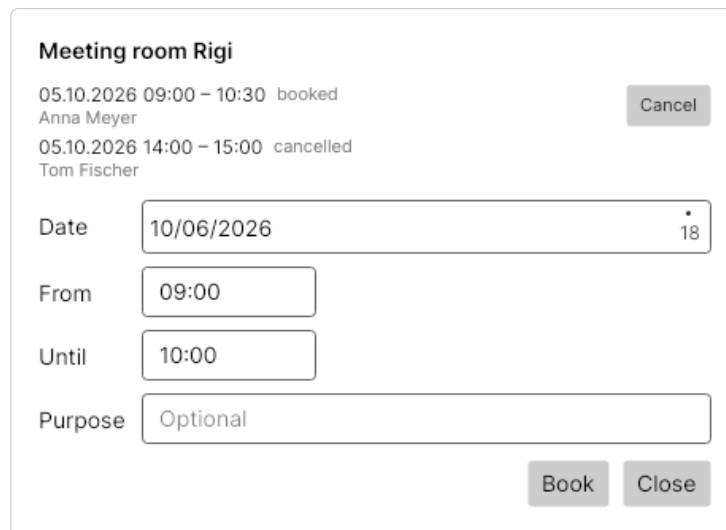
- **My tasks** — the reviews assigned to you, as to-do items. This is what a reminders or task app subscribes to.
- **My task deadlines** — the same reviews as dated entries, for calendar programs that do not show to-dos.

Both are **read-only**. They are a view of the **My tasks** tab, published so you can see what is waiting for you without opening the archive; approving or rejecting a review still happens in the workbench, where it is recorded with who did it and why.

A review only gets a deadline if its document type defines one. The review SLA is set per document type, and where none is set the review simply has no due date — it still appears in **My tasks**, but it cannot appear in **My task deadlines**, which is a list of dates. An empty deadlines list usually means no document type in your tenant has an SLA configured.

7.5 Reserving a meeting room

A **meeting room** is a folder whose type is *Meeting room* — an administrator creates one wherever rooms should live, and whoever may see the folder may see its reservations. Select a room and the ribbon offers **Bookings...**: the reservations that exist, and a row to take a new one.



Meeting room Rigi

05.10.2026 09:00 – 10:30 booked
Anna Meyer Cancel

05.10.2026 14:00 – 15:00 cancelled
Tom Fischer

Date 18

From

Until

Purpose

Book Close

Figure 14: The Bookings dialog on a room: existing reservations above — a cancelled one stays visible as history — and the new reservation below, its times typed rather than picked.

Pick the date, type the times — 0950, 9:50 and 09:50 all work — add a purpose if you want one, and **Book**. A slot that overlaps an existing reservation is **refused**, and the refusal says so: the room is never silently double-booked, and there is no queue to wonder about. **Cancel** keeps the row, marked cancelled, so “who had the room booked?” stays answerable; cancelling frees the slot for the next person.

A reservation can repeat. The dialog books single slots; a weekly jour fixe is made as a repeating calendar entry in the room’s Schedule — from the Calendar tab or from your phone — and **every occurrence** is conflict-checked, so a series that would collide with anything on any of its dates is refused as a whole. The one condition: a repeating **reservation** must have an end (a last date or a number of times). “Every Monday, forever” cannot be checked against the next endless series, so the archive does not accept it.

Behind the dialog there is no separate booking system. The first reservation creates a **Schedule** calendar under the room, and every reservation **is** a calendar entry in it — a document like any other, with versions, an audit trail and the room’s own permissions. Open the Schedule in the tree and you are looking at the same reservations the dialog shows; tick it in the **Calendar** tab and they overlay your own appointments, colour-coded like any other collection.

And because the Schedule is a calendar, your phone can subscribe to it — the same CalDAV subscription as any other calendar (Section 7.3). That cuts both ways, deliberately:

- **Reading:** tick the room’s Schedule on your phone and the room’s reservations appear beside your own appointments — the fastest answer to “is the room free at three?” is the calendar already in your pocket.
- **Writing:** creating an event in that calendar from your phone **is** booking the room, and it passes the **same** conflict check as the dialog. Dragging a reservation to another time is a rebooking, checked again; there is no way in — workbench, phone, or any calendar program — that bypasses the rule.

7.5.1 Three calendars, three different questions

A bookable thing — a room, a vehicle, a person’s time — answers three questions about the same timeline, and each gets its own calendar because each obeys a **different rule**. Putting them in one calendar would mean one rule for three jobs, and two of the three would be wrong.

Calendar	It says	And the rule that makes it different
Schedule	This time is <i>claimed</i>	Two reservations may never overlap. A slot that collides is refused, and the refusal says so.
Maintenance	This time is <i>withdrawn</i>	Blocks may overlap each other — two defects reported on one vehicle are two entries, and refusing the second would mean the second finding cannot be recorded.
Availability	This time is <i>offered</i>	Windows may overlap each other, and reservations sit inside them — that overlap is the whole point, not a conflict.

Out of service. A block filed in Maintenance (“projector broken, Tuesday to Thursday”) **suspends** every reservation it overlaps rather than deleting them; when the block is cleared, those reservations simply stand again. Nothing is stored as “suspended”, so nothing can be left stale, and nobody can edit a reservation back into service — the only way out is clearing the block, which is a right of its own. A suspended reservation shows on your phone as **tentative**, so you can see it is in doubt without it vanishing from your week.

The two moments differ, deliberately. A **new** booking that lands in a block is **refused** at the door; a booking that already existed when the block appeared is **suspended**. The archive will not let you walk into a known problem, and it will not quietly cancel a commitment you already made.

Offered time. An Availability window is somebody publishing when a thing may be booked — an instructor’s teaching hours, a pool car’s lending times. Two rules about it are worth knowing because the obvious guess is wrong in both cases:

- **A window must cover the whole booking, on its own.** Two adjacent windows do not add up: 09:00–11:00 and 11:00–13:00 will not between them carry a 10:00–12:00 booking. Accepting half an offer as consent for a whole booking is a failure nobody would see — the booking succeeds, somebody is committed to a time they never offered, and the first anyone learns of it is when they do not turn up.
- **No window means not offered.** A thing nobody has published time for offers nothing. Absence of an offer is not consent.

Whether a reservation **requires** an offer at all depends on what is being booked, and that is decided by the industry module that owns the thing rather than by the archive: booking an instructor may need their published hours as their consent, while booking a meeting room needs nothing of the sort. The archive stores the offer and answers the question; what the answer **obliges** is the module’s rule.

7.6 On a phone or a tablet

The same address, in the same browser, gives you a layout built for the screen you are holding. There is no separate mobile site and nothing to install — and nothing is **taken away**: what the narrower layouts cannot show side by side, they show one at a time.

A tablet is recognised by its touch screen, not by its width. That distinction matters more than it sounds: a large tablet is about 1024 points across upright and 1366 across sideways, so measuring width alone would call the same device a tablet one way up and a desktop the other. Turning your tablet changes how many panes you get, and never which kind of device the archive thinks you have.

7.6.1 A phone: one pane, and a way back

The folder tree becomes a drawer, the folder's contents fill the screen, and a document opens over the top with its own **Preview**, **Details** and **Comments** tabs — so the panes the wide layout drops are a tap away rather than gone.



Figure 15: A folder's contents on a phone, filling the screen. A single tap opens — there is no double-click here.



Figure 16: The tree, slid in from the left. Choosing a folder closes it again.



Figure 17: A document, full screen, with its three tabs. Back returns to the list.

A single tap navigates on a phone. On a desktop a click selects and a double click opens, because there is a second pane to show the selection in. On a phone there is not, so a tap goes straight in.

7.6.2 A tablet: one pane upright, two sideways

Held upright, a tablet behaves exactly as a phone does — one pane, the tree in a drawer.

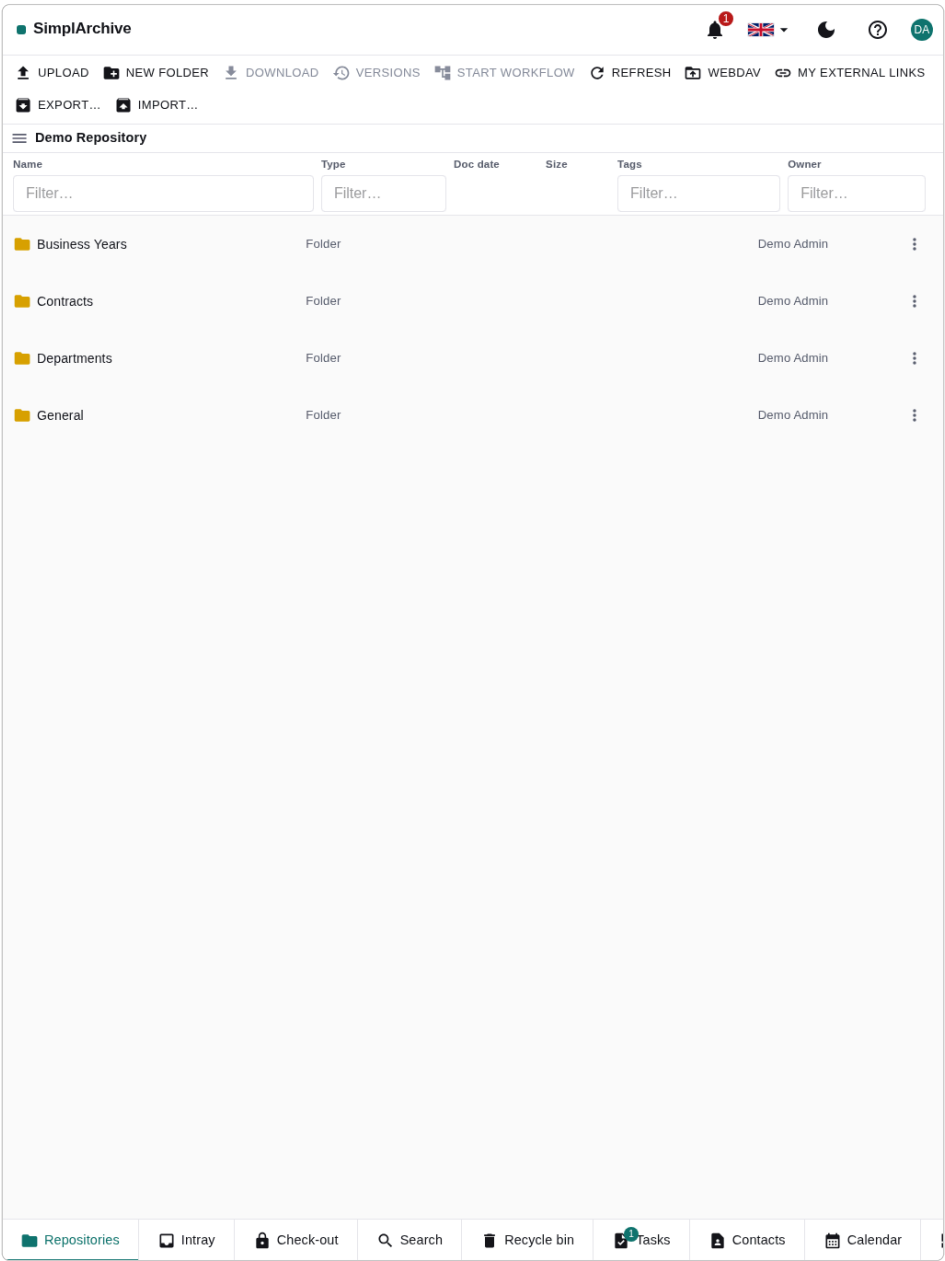


Figure 18: Upright: one pane at a time, the same shape as a phone.

Turn it sideways and you get two of the three panes. While you are browsing, the tree sits beside the folder’s contents.



Figure 19: Sideways, browsing: the tree beside the contents list.

Open a document and the tree steps aside for it, so what you came **from** stays next to what you opened – and the tree is one tap away from the ≡ button whenever you want it back.

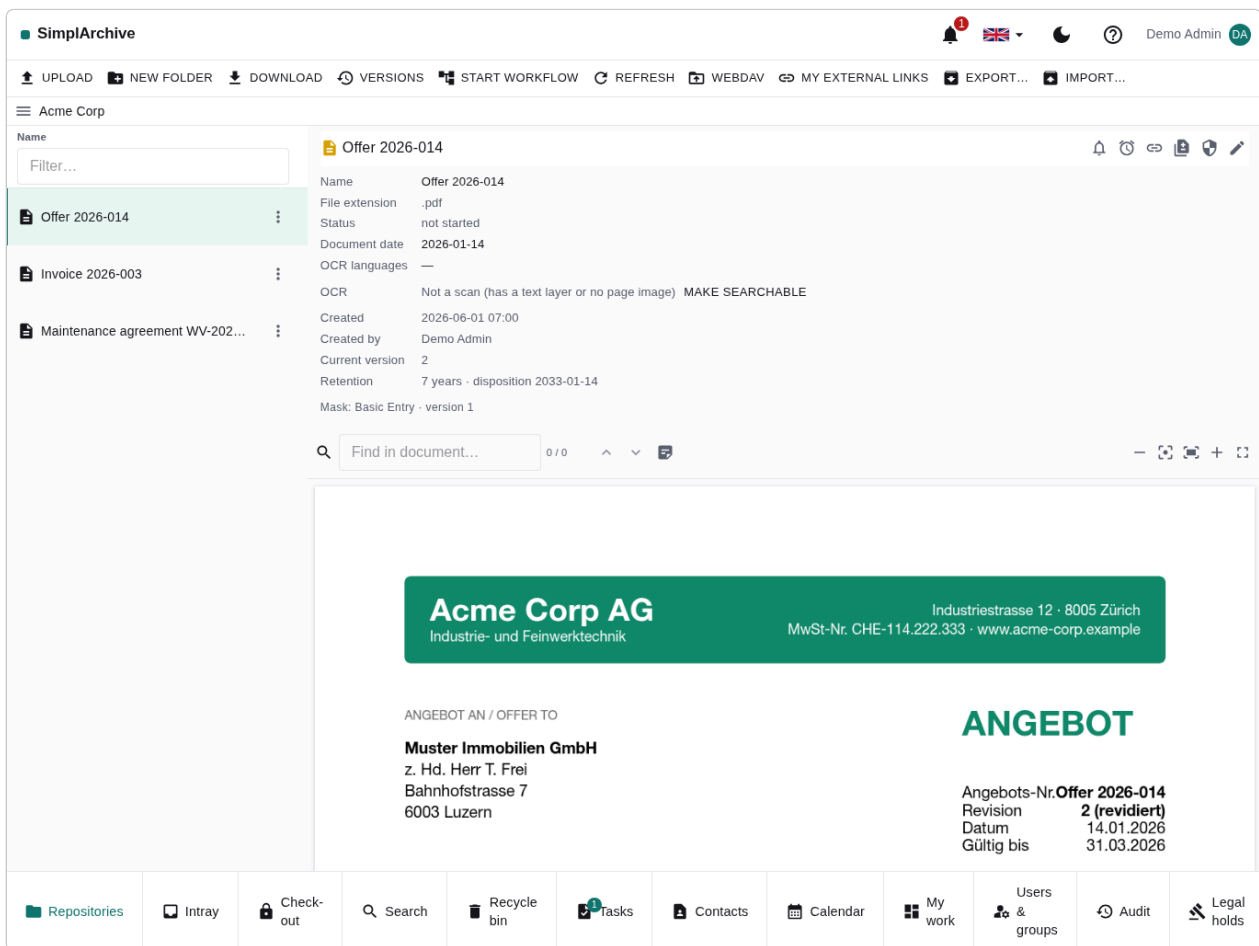


Figure 20: Sideways, with a document open: the list beside the document. The tree is a tap away.

7.6.3 What each width shows

Screen	What you see
Desktop	All four panes — tree, contents, document, comments — resizable, and remembered per browser
Tablet, sideways	Two panes: tree and contents, or contents and document
Tablet, upright	One pane, tree in a drawer
Phone	One pane, tree in a drawer, document full screen

The comments pane is the first to go as the screen narrows, and the index-data pane the next — both come back as tabs inside the document view, so nothing becomes unreachable. It moves.

Buttons are labelled where you cannot hover. On a desktop a ribbon button is an icon and its name appears on hover. A touch screen has no hover and therefore no tooltip, so the same buttons keep their labels — an unlabelled symbol you cannot interrogate is a guess.

This chapter is about the archive **in a browser**. For your device's own mail, contacts, calendar and files apps reaching the archive with no browser at all, see Section 8.

8 Your own device

Everything in this chapter is already yours. SimplArchive installs **nothing** on your phone, tablet or laptop — the archive speaks the protocols those devices came with, so the mail app, address book, calendar and file manager you already use are the client. There is no app to find, no version to keep current, and nothing to uninstall when you leave.

That also means the archive obeys the same rules on your device as in the workbench. What you may see is what you may see; a folder you have no rights to simply is not there, rather than being there and refusing you.

8.1 What speaks what

On your device	Protocol	What you get
Mail program	IMAP	The archive as mail folders — see Section 6.2
Address book	CardDAV	Contacts sync both ways — see Section 7.3
Calendar	CalDAV	Calendars and your task deadlines — see Section 7.3
File manager	WebDAV	One mountable drive — see Section 6
Notes app	IMAP	Notes with real version history — see Section 6.2
Browser	—	The full workbench, on any screen

8.2 One password for your devices

Every protocol above uses the **same DAV password**, issued from **WebDAV...** in the account menu. It is separate from your sign-in password on purpose: a device holds it indefinitely, so it is the one you revoke when a phone is lost — without changing how you sign in.

Revoking it disconnects every device at once. There is no per-device credential today; if you need one device cut off, reissue and re-enter the password on the ones you keep.

8.3 Encryption is not optional

Point your devices at an `https://` address, and make sure the mail account uses **TLS** (port 993). This is not only good practice — some notes clients **silently refuse** a plaintext connection: they ask the server what it supports, dislike the answer and hang up, showing you nothing at all. A server that works perfectly from every other app can look broken from that one, and the reason never appears on screen.

If a device connects but shows nothing, suspect the transport before the account. A wrong password produces an error you can read; a refused plaintext connection produces silence.

8.4 What each one does not give you

Naming the boundary is what stops a working feature looking broken:

- **A mail program shows the archive, not the application.** No recycle bin, no workflow, no permissions dialog, no search across metadata — those live in the workbench.
- **A mounted drive shows the tree, not the history.** Versions, comments and index data are not files, so they do not appear; saving over a file makes a new version, which is the one piece of history the drive can express.
- **Calendars and address books carry their own items only.** A calendar holds appointments; it will not show you the documents filed beside it.
- **Task collections are read-only.** You can see what is waiting; approving it is recorded in the workbench, with who did it and why.

8.5 On a small screen

The workbench itself adapts rather than sending you to a separate mobile site, and it decides by what you are holding rather than by how wide the window is — a tablet is recognised by its **touch** screen, so turning it does not turn it into a laptop.

- **Phone, and a tablet held upright** — one pane at a time. The folder tree slides in from the left, the folder contents fill the screen, and opening a document covers it with **Preview**, **Details** and **Comments** tabs. A single tap navigates, rather than the desktop's click-then-double-click.
- **Tablet held sideways** — two panes. Browsing shows the tree beside the folder contents; opening a document swaps the tree for the document, so what you came from stays next to what you opened. The tree is always one tap away, from the ☰ button.
- **Laptop or desktop** — the full workbench, with resizable panes that remember their widths.

9 Metadata & classification

Each document has a **mask** (document type) that defines its **index fields** — typed metadata such as an invoice number or a date. Open the **detail** pane's edit toggle to fill them in. A document also carries a **document date** and one or more **OCR languages** used to make scans searchable.

Index fields are validated by the mask — a field marked required must be filled, and format/range rules are enforced when you save. Well-known masks (Folder, Basic Entry, e-Mail) exist in every tenant; administrators can add more.

10 Search

The **Search** tab runs a full-text search across document **content**, **names**, and **index-field** values, ranked by relevance, with hits highlighted on the preview — so a distinctive word buried in a document’s text (a product name on an invoice, say) finds exactly that document. **Refinement filters** and **facets** narrow the results — by document type, date, sensitivity, and more — and you can **save** a search to re-run or share it later.

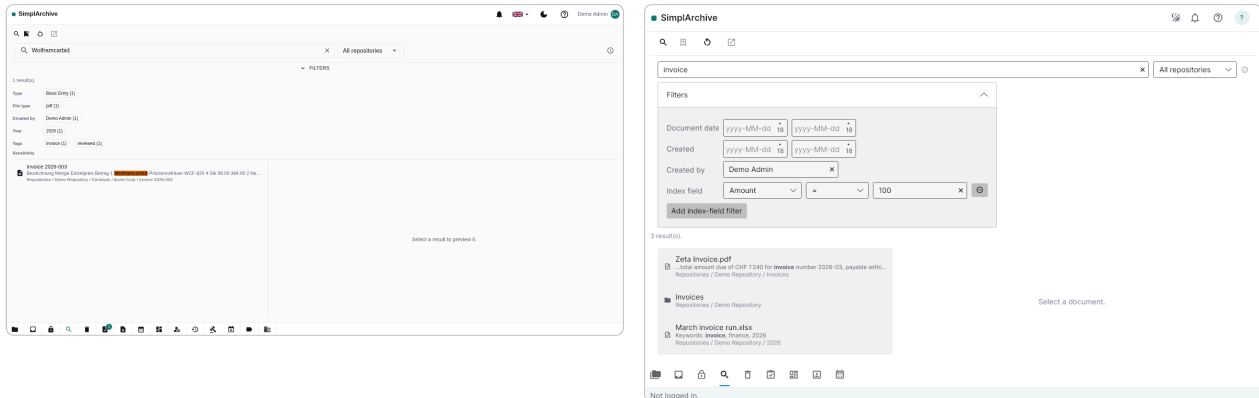


Figure 21: Full-text search with refinement filters and facets, in the web (left) and desktop (right) clients.

Parts of words. Search matches whole words first, which is what keeps the most relevant document at the top. If that finds nothing, the search is retried looking for your text **inside** words — so **montage** finds a document that only ever says **Montagehalterung**, and **sechskant** finds the longer compound containing it. This matters most in German, where one word does the work of several. The second attempt is slower and its ranking is flatter, so it only runs when the first found nothing at all.

What the field does not do. Operators are not interpreted: quotes, AND, **field:value** and wildcards are searched as literal text, deliberately — a query can never fail over a stray character. Structured narrowing — by document type, owner, year, tags, or a specific index field — lives in the refinement filters beside the results, not in the text box.

11 Collaboration

Documents are collaborative: post **comments** in a feed, attach **annotations** (sticky notes, highlights, and shapes) onto the preview, and **follow** a document or folder to be **notified** of activity. Set yourself a **reminder**, and track everything assigned to you on the **My work** dashboard. To edit a document exclusively, **check it out** — others see it is locked — then **check in** your changes as a new version.

The chat thread. Every document has its own thread, shown beside the preview. Use it for the conversation about the document — a question about a figure, a note that a revision is on its way — so that discussion stays with the document instead of in someone’s mailbox. Replies nest under the message they answer, and the thread also records what happened to the document: a new version, a filing, and other activity appear in the same timeline, so reading it tells you both what people said and what changed.



Figure 22: The chat pane on a document: a threaded conversation interleaved with the document’s own activity — here a question and its reply, followed by the two versions as they were saved.



Figure 23: Left: the **My work** dashboard. Right: the **Check-out** tab listing documents locked for exclusive editing — here one edited (*Modified*) and one untouched (*Unchanged*).

Seeing your edit before you commit it. The **Check-out** tab shows what you are **about to check in**, not what is already archived. Select a document you hold and the pane beside the list shows its index data and a preview of your **working copy** — the file as you have edited it. That is the version the **Check in** action will create, so you can look at it before committing, without leaving the tab to go and find the archived copy (which would show you the one thing that is certainly not your edit).

A document you have edited is marked **Modified**, and only those offer the actions that make sense for an edit:

Action	What it does
Compare	An inline, side-by-side view of what changed between the archived version and your working copy.
Beyond Compare	Opens the same two files in Beyond Compare, if you use it — a desktop-only convenience, since a browser is not allowed to launch another application. Not affiliated with SimplArchive; the button points you to the vendor if you do not have it.
Check in	Files your working copy as a new version and releases the lock.
Discard	Throws the working copy away and releases the lock, leaving the archived version untouched.

If you have not saved anything to your working copy yet, the preview stays empty rather than showing the archived version. There is genuinely nothing to preview, and showing the archived file there would be answering a different question.

12 Sharing outside SimplArchive

Everything above assumes the other person has an account. An **external link** is for when they do not: a plain URL that opens one document, for anyone who has it, with no sign-in. Use it for the customer who needs the signed contract or the auditor who needs one invoice — not as a general-purpose way to move documents around.

Creating one. Select the document and open **External links...** from the detail pane. Choose when it should expire and how many times it may be opened, then create it.

The URL is shown once, at creation, and never again. Copy it before you close the dialog. It appears in no list afterwards — deliberately, because a list is read far more widely, and far more casually, than the moment you deliberately created something.

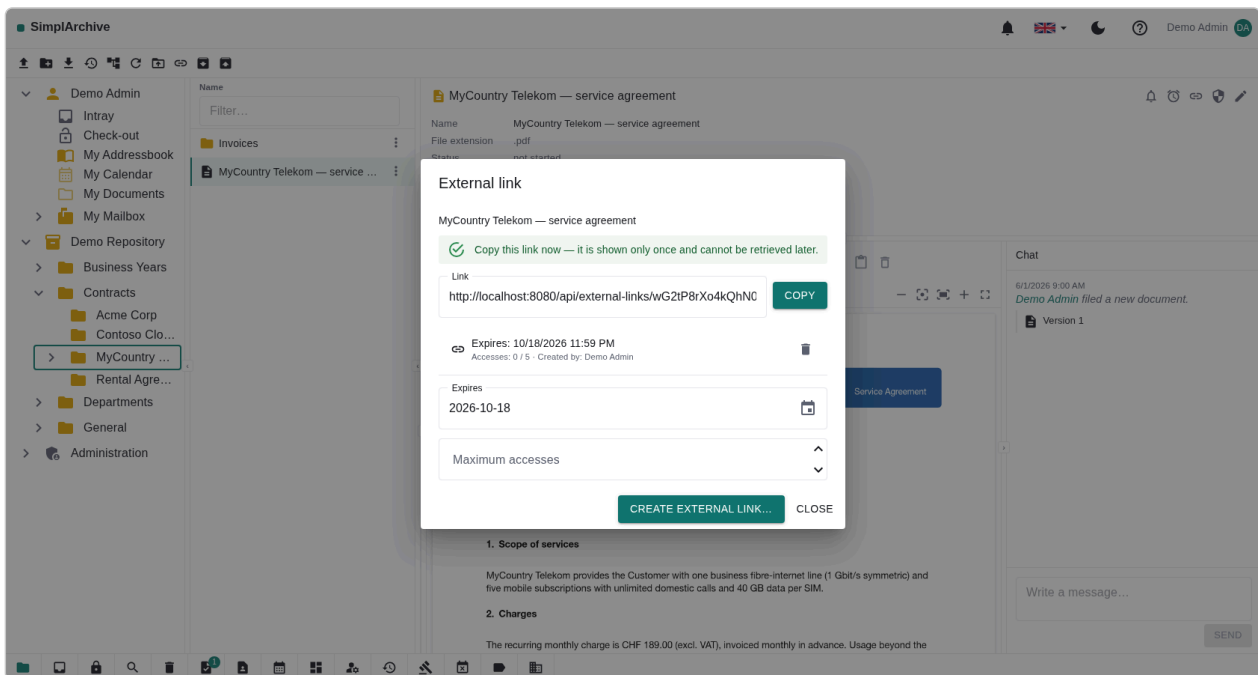


Figure 24: Creating an external link. The URL is revealed once, here, and cannot be retrieved later.

Managing what you have shared. My **external links** on the ribbon lists your live links: which document, when it expires, how many times it has been opened. From a row you can jump to the document, review a link's details, or **revoke** it. An administrator sees everyone's, filtered by user or group.

A link near its expiry can be **extended** — but only within 30 days of lapsing, by at most 90 days, and measured from today rather than added to whatever time remains. So extending is a deliberate renewal, not a way to accumulate an indefinite link.

Revoking does not delete the row. It stamps it revoked and leaves the record standing — who shared what, with effect from when. After a link has leaked, that record is exactly what the investigation needs, and it would be gone if revoking tidied it away.

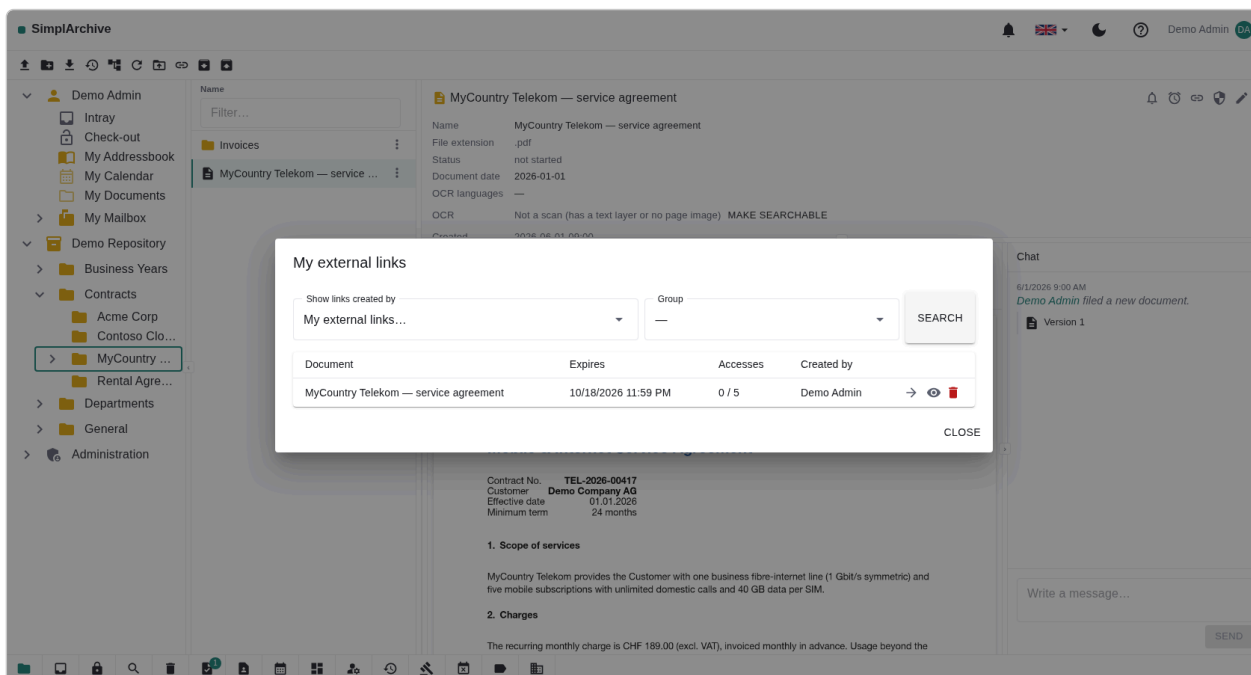


Figure 25: **My external links** — every live link you have shared, with go-to, details and revoke on each row.

What the recipient sees. A single page with the document's name, a picture of its first page, and buttons to open or download it — with the number of pages marked on the picture when there is more than one, so they know what they are about to open. Nothing else: no tree, no navigation, no route to any other document.

The picture is drawn when you create the link, not when the recipient opens it, so there is nothing to wait for. A document with no picture to draw — an archive, a binary — simply shows its name and the buttons.

An unknown, expired, exhausted or revoked link all produce the **same** response. That is deliberate — telling a stranger which of those they hit would confirm a real link exists and hint at how to reach a usable one.

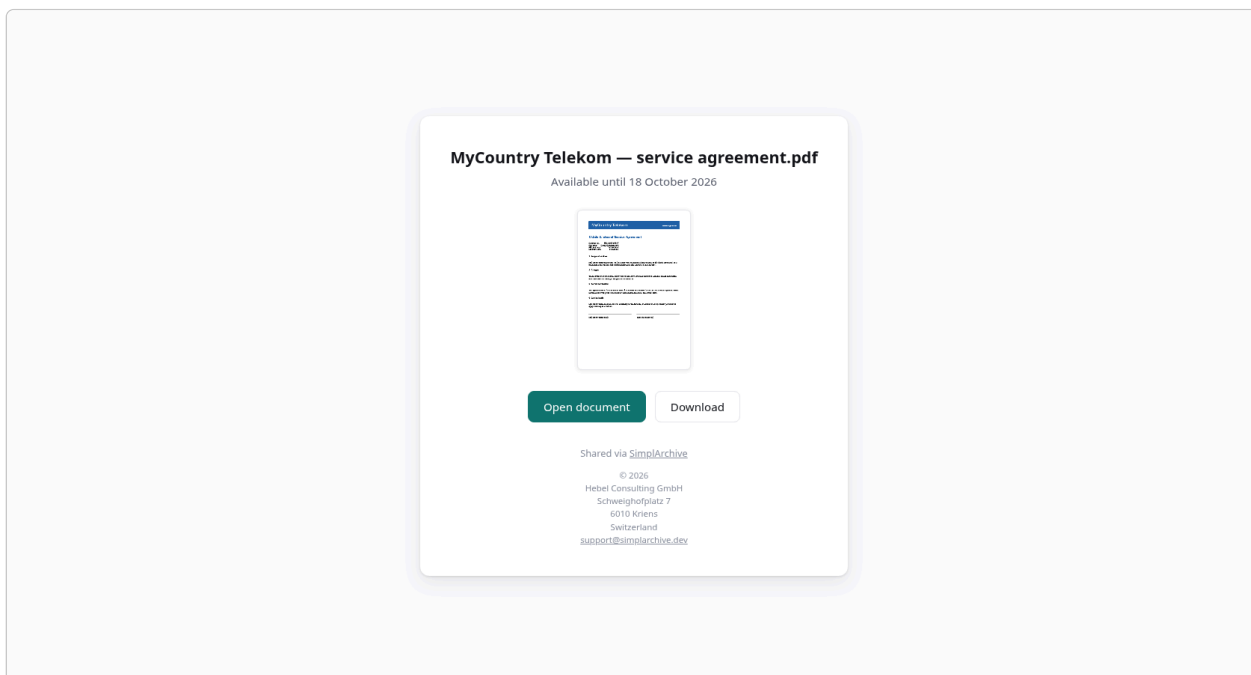


Figure 26: The recipient's view: the document's name and a picture of its first page. This one is a single page, so it carries no page-count marker. No account, one document, nothing else reachable from it.

12.1 The controls an administrator holds

Sharing outward is the one action that leaves the system, so it is gated twice and can be stopped in one move.

- **Two gates on creating a link.** The tenant-wide **Allow external links** switch must be on, and the individual needs the **Create external link** right, granted per user or group from **Users & groups**. Either alone is enough to prevent sharing; both must be present to allow it. A service account can never create one — only a person shares, so there is always someone accountable for a link's existence.
- **The kill switch.** Turning **Allow external links** off is checked when a link is **opened**, not only when one is created — so it stops every link already out in the world, not merely future ones. This is the control to reach for when something has leaked, and it is worth knowing about before you need it.
- **The caps you set.** **Maximum link lifetime** (default 180 days) and **default access count** (default 5) are the rails everyone else shares within. Tighten them to your own policy rather than relying on people choosing well.

13 Workflow & records

Approval workflow. Submit a document for review and it moves through a fixed state machine — Draft → In Review → Approved / Rejected → Released. Reviewers act on their **Tasks** tab — sortable by any column (a click on a header; the default puts the nearest deadline on top, overdue in red first) and narrowable through the visible filter row: document and version text, plus an **Overdue only** switch. Reviews can be reassigned, and overdue reviews escalate.

Records management. A **legal hold** freezes documents so they cannot be changed or deleted. **Retention** policies dispose of documents once their retention period ends (with review before disposition, if required). Deleted documents rest in the **Recycle bin**, from which an administrator can restore them or purge them permanently.



Figure 27: The Tasks tab — a reviewer’s approval queue — in the web (left) and desktop (right) clients.



Figure 28: Records management: **legal holds** (left) freeze documents; **retention** (right) governs disposition.

14 Administration & account

Administrators manage **users & groups** and the **rights** granted to them, configure **tenant** settings, and review the tamper-evident **audit trail** of every security-relevant action. They also curate catalogues (sensitivity labels, tags), set the **storage quota**, and run **import / export**. Every user manages their own **account security** — password, multi-factor authentication (authenticator app or passkeys), and profile photo.

Your own account lives behind **Edit profile...** in the avatar menu (top right in both clients). It shows which account you are signed in as, the photo you currently have with a crop to replace it, and a button through to changing your password. Two-factor authentication, passkeys and the WebDAV password stay as their own entries in that menu, since each is a separate credential rather than part of your profile.



Figure 29: Users & groups administration in the web (left) and desktop (right) clients.

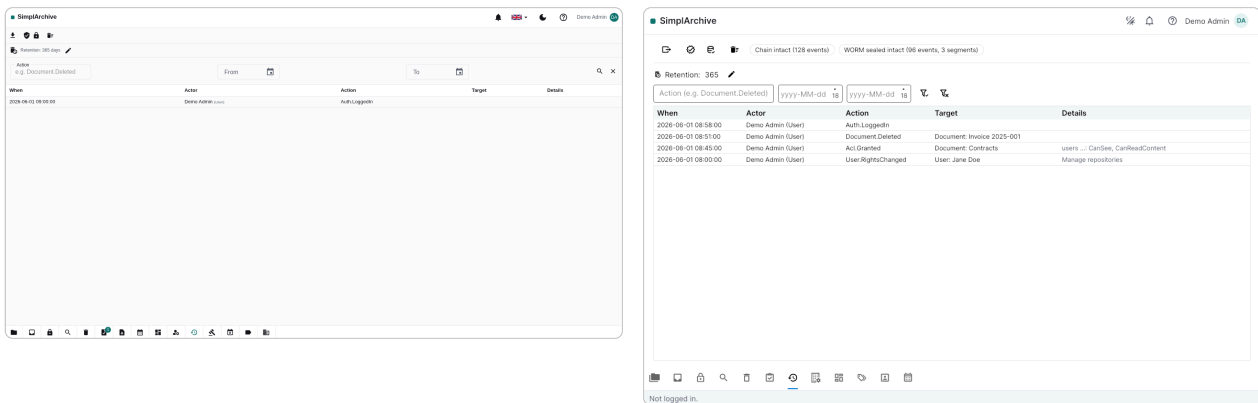


Figure 30: The audit trail — an append-only, hash-chained log — in the web (left) and desktop (right) clients.



Figure 31: Tenant settings in the web (left) and desktop (right) clients.

The desktop client can connect to **several servers** its server manager stores a profile (name + address) for each. A server is not a tenant: one SimpliArchive installation hosts many tenants, and which tenant you belong to follows from the account you sign in with.



Figure 32: The desktop server manager: connection profiles for several SimplArchive servers.

15 Appendix — glossary & links

Repository — a root folder (a document with no parent). **Mask** — a document type / set of index fields. **ACL** — per-document access-control list. **Version** — an immutable snapshot of a document's file. **Legal hold** — a freeze that blocks change/deletion. **Retention** — a policy that disposes of documents after a period. **vCard** / **iCalendar** — the standard file formats a contact and a calendar entry are stored in (.vcf / .ics). **CardDAV** / **CalDAV** — the standards a phone or mail program uses to sync addressbooks and calendars. **UID** — the identifier those programs use to recognise an item across devices; changing it makes a duplicate rather than an edit. **SLA** — the review deadline a document type sets, which is what gives a task a due date.

Further reading:

- The live API description — the OpenAPI document at `/openapi/v1.json`.
- Desktop-client downloads — the download area at `/download`.

16 Appendix: the desktop client's log

When the desktop client misbehaves — a preview that stays empty, a server that will not connect, a file that does not open — its log usually says why. The client writes one rolling log file per day, always at full detail, so there is nothing to switch on after the fact: whatever just went wrong is already recorded.

Where to find it. The easiest way is in the app itself: **Help ► Show log folder** opens the folder directly. The location by operating system:

- Windows — %APPDATA%\SimplArchive\logs
- macOS — ~/Library/Application Support/SimplArchive/logs
- Linux — ~/.config/SimplArchive/logs

Files are named `simplarchive-YYYYMMDD.log`; the newest one is today's. The log is small by design (it rolls at 4 MB and keeps seven files) and never contains a password or a token, so it is safe to attach to a support request as it is.

Watching it live. Start the client from a terminal with the `--verbose` flag and the same full detail is printed to the terminal as it happens — useful when reproducing a problem step by step:

- Windows — `SimplArchive.DesktopClient.exe --verbose`
- macOS — `SimplArchive.app/Contents/MacOS/SimplArchive.DesktopClient --verbose`
- Linux — `./SimplArchive.DesktopClient --verbose`

Without the flag a terminal shows only the important lines; the file always carries everything, flag or no flag.

17 Appendix — who does what

A short inventory of what each kind of participant can do, for readers who need the shape of the system before its detail: an evaluator sizing it up, an administrator planning a rollout, or anyone deciding which chapter of this manual they actually need. Everything here is described properly in the chapters above; this is the map, not the territory.

Everybody's rights are per document. None of the lists below overrides that. What a person may do to a particular document comes from that document's permissions (and the groups they belong to), inherited down the tree unless a folder breaks the inheritance. An action missing from the interface is usually not a missing feature — it is the server declining to offer it here, to you, now.

17.1 The everyday user

Find and read	Browse the tree, open a folder, preview a document without downloading it, search by content or index data, jump to a search hit inside the page, and follow references to wherever else a document is filed.
Add and change	Upload by drag or from the ribbon, add a new version to an existing document, stage work in the Intray until it is ready to file, check a document out and back in , and edit index data.
Organise	Create folders, move documents, place a reference so one document appears in a second folder, tag, and classify with a document type (mask).
Collaborate	Comment in a document's chat and mention colleagues, annotate a page, subscribe to a document to hear about changes, set a reminder, and act on the tasks a workflow sends you.
Share	Create an external link for someone without an account — time-limited and access-limited — and revoke it.
Work elsewhere	Mount the archive in the operating system's file manager, sync contacts and calendars to a phone, read the archive's mail folders in a mail client, and open a document in its native desktop application from the desktop client.
Their own account	Language and appearance, profile photo, password, two-factor authentication and passkeys, and the separate app passwords their devices use.

17.2 The tenant administrator

Everything above, plus the settings that govern their own organisation. A tenant administrator can reach every document in their tenant: the bypass is deliberate, and what they **do** with it is audited like anything else.

People	Create and deactivate users, build groups, grant the system rights that let somebody administer users, view the audit log, import, or manage records — and issue service accounts for integrations.
Document types	Define masks and their index fields, which folders they may live in, what they may contain, their retention period and their default sensitivity label.
Records	Place and lift legal holds , review documents that have reached the end of their retention before they are disposed of, and purge when that is the decision.
Oversight	Read, filter, export and verify the audit log , and stream it to a SIEM over a signed webhook.

Policy	Require multi-factor authentication for everybody, cap storage, allow or forbid external links and bound their lifetime, restrict tags to a catalogue, and set the retention and check-out defaults.
---------------	--

17.3 The platform administrator

Runs the **installation** rather than any one organisation in it. They create and suspend tenants and provision the first administrator of each. They deliberately belong to no tenant, so they do not see documents; the separation is the point, and it is why a platform administrator cannot quietly read a customer's archive.

17.4 The service account

A machine identity for an integration — a scanner, a line-of-business system, a script. It authenticates with a client id and secret rather than a password, holds exactly the rights it was granted (and never more than the person who granted them held), and appears in the audit log under its own name. It is the supported way to put documents in automatically, and it is why nobody needs to share a human's credentials with a robot.

17.5 The recipient of an external link

Somebody with no account at all, holding a URL. They can open the one document it names — and no other — until the link expires or its permitted number of uses runs out. Nothing else in the archive is reachable from it, and the link can be revoked the moment it turns out it should not have been sent.

17.6 Programs, not people

A file manager	Mounts the archive over WebDAV and shows exactly the tree the workbench shows. Opening, editing and saving a file works the way it does on a network drive.
A phone or a calendar program	Syncs contacts and calendars over CardDAV and CalDAV. What it changes, the archive keeps — and versions.
A mail client	Reads the archive's mail folders over IMAP, and can file a message into the archive by copying it there.
A mail server	Delivers mail straight into a user's Inray, so an address can be filed simply by being written to.

All four use **app-specific passwords**, issued from the account menu and revocable one at a time — never the password used to sign in. A lost phone costs one revocation, not a password change.

18 Appendix — how SimplArchive is put together

The vocabulary the system actually thinks in. Worth ten minutes if you are evaluating it, integrating with it, or wondering why something behaves the way it does — several of the answers in this manual are consequences of the shape below rather than of any particular screen.

18.1 The document is the spine

Almost everything is a **document**. A document has a name, a place in the tree, permissions of its own, and — usually — content.

A folder	A document with no content. That is the whole difference: folders are not a separate species, which is why a folder has index data, permissions, a chat thread and a history like anything else.
A repository	A document with no parent — a root. Again not a separate species, which is why the same permission rules reach the top of the tree without a special case.
Your own space	A repository belonging to one person, provisioned for them, holding their Intray and their checked-out documents. It is why every user has somewhere to put something before deciding where it belongs.

18.2 Content lives in versions

A document's file is a **version**, and versions are immutable. Uploading again adds a new one; the old one stays readable. This is what makes “restore an old version” honest — nothing is overwritten, so nothing has to be recovered.

Metadata belongs to the document, content belongs to the version. Rename a document, re-classify it or edit its index data and the history is untouched; upload a new file and the metadata carries across. That split is why re-filing a document never costs its history, and why a version can be restored without copying anything.

18.3 Document types and index data

A **mask** is a document type: a named set of **index fields** with their types, validation and defaults. Assigning a mask to a document decides what may be recorded about it, and — through the mask — where it may live and what it may contain.

Masks are **versioned and immutable**. Changing one publishes a new version; documents already classified keep the version they were classified under. A field's meaning therefore never changes retrospectively, which is what makes a ten-year-old document still readable on its own terms.

18.4 Who may do what

Permissions are an **access-control list per document** — a set of rights granted to a user, a group or a service account. Rights inherit down the tree until a folder is set to break inheritance, at which point that subtree starts again from what is granted there.

Two rules are worth knowing because they explain refusals:

- **Nobody can grant a right they do not hold.** An administrator of a folder cannot manufacture access to it for someone else beyond their own.
- **A deactivated user and a suspended tenant have no rights at all**, decided before any administrator bypass — so deactivating an account is immediate and total rather than a matter of stripping grants one at a time.

18.5 One installation, many organisations

A **tenant** is an organisation. Every row that belongs to one carries its tenant, and the isolation is enforced in a single place — a filter the database context applies to every query — rather than by each feature remembering. Users, groups, masks, documents, tags and audit events are all tenant-scoped; a **platform administrator** deliberately belongs to no tenant.

18.6 The things that hang off a document

Chat, mentions	The conversation about a document, and the system entries that record what happened to it — a version filed, an older one made current, an attachment refused.
Annotations	Marks on a page, anchored to the version they were drawn on.
References	The same document appearing in a second folder. One document, two places — not a copy, so there is nothing to diverge.
Tags	Free or catalogue-restricted labels, orthogonal to the tree.
Subscriptions, reminders	Who wants to hear about changes, and when somebody wants to be reminded.
External links	Time-limited, use-limited access for somebody with no account.
Legal holds, retention	A freeze that blocks change and deletion, and a policy that eventually disposes of what is no longer needed. A hold always wins.
Workflow	The state a document is in, and the trail of transitions that got it there.
Audit events	What was done, by whom, to what — hash-chained so a gap or an edit is detectable.

18.7 What the archive stores it in

Content goes to object storage, never through the application: the browser and the desktop client transfer bytes directly against a short-lived signed URL. Metadata, permissions and the audit trail go to PostgreSQL. Full-text search is a separate index built from the extracted text, and the archive works without it — search falls back to metadata alone rather than failing.

Nothing above is peculiar to the web client or the desktop client. Both are views onto the same model, which is why a document filed from a phone's calendar, a mail server, a file manager and the workbench are the same kind of thing afterwards — and why the permissions on it mean the same thing in all four.

19 Appendix — security posture

A self-assessment against the **OWASP Top 10 (2021)** and a summary against **ASVS Level 2**, for readers who have to answer for what they deploy.

What this is, and is not. It is a structured review of the software by the people who wrote it, kept current with the code. It is **not** a penetration test and not a third-party certification, and it says nothing about any particular installation's network, backups or operating procedures. The distinction the project holds elsewhere applies here too: the product is production software; the public demonstration instance is not, and is documented as such.

19.1 The finding that mattered

The review's most useful result was not any single weakness. It was a **pattern**: the four gaps it ranked highest had all been **decided, written down, and never built**. Response headers, sign-in throttling, an outbound-request guard and an upload content check each had an architecture record specifying them — in one case naming the exact four controls — and none of the four existed in the code. Nothing was visibly broken, which is precisely why they survived: the absence of a preventive control is invisible until the day it is needed.

All four are now built, each with its own record explaining what was decided and what it concedes. The lesson is kept: a decision recorded is not a decision delivered, and only a test can tell the two apart.

19.2 OWASP Top 10 (2021)

A01 Broken access control	Permissions are an access-control list per document, resolved by one calculator that every caller goes through — with group expansion, inheritance and explicit breaks. Nobody can grant a right they do not hold. A deactivated user or suspended organisation has no rights at all, decided before any administrator bypass. Multi-tenant isolation is a query filter applied in a single place rather than remembered per feature, and a test suite asserts a tenant cannot reach another's data.
A02 Cryptographic failures	Passwords are stored only as hashes, and never logged. Secrets — database credentials, object-storage keys, token-signing certificates — come from a secrets manager when one is configured, with the database credential issued afresh at every start-up and the application never using the database superuser. Transport security is relaxed only when the application is run in development mode: the relaxation is tied to that mode rather than to a setting, so there is no configuration switch that turns it off in a deployed installation.
A03 Injection	Data access is exclusively through a typed query layer: the application contains no raw or string-concatenated SQL anywhere outside its schema migrations . Output in both clients is framework-escaped, and the response-header policy blocks inline script rather than allowing it. XML from the protocol

	endpoints is parsed with document-type definitions disabled by framework default.
A04 Insecure design	Progressive throttling of credential guessing at every door that verifies one; an upload content check that refuses executables and scripts by their bytes as well as their names; storage quotas; connection caps on the mail endpoint; page-size clamps on every listing. The archive's own invariants — no cycles in the tree, no duplicate sibling names, immutable document types — are enforced at the single point where data is saved, not by each caller.
A05 Security misconfiguration	A start-up gate refuses to run outside development when any development-grade setting is present — development certificates, bootstrap secrets, default storage credentials, an unstamped build — and there is no flag to bypass it . Responses carry a content-security policy, frame and referrer restrictions and content-type pinning; each header is set only if absent, so a deployment that owns its own edge keeps ownership.
A06 Vulnerable and outdated components	Every build scans the dependency tree for known vulnerabilities and fails on a finding; the container image is scanned separately; dependency updates are proposed automatically each week. A licence gate additionally fails the build on any dependency outside the allowed set.
A07 Identification and authentication failures	Standards-based sign-in with proof-key exchange for the interactive clients. Second factors are supported as time-based codes and as passkeys, including passwordless sign-in, and an organisation can require them for everybody. Failed attempts are progressively refused per account and per source, with blocks that expire on their own rather than needing an administrator.
A08 Software and data integrity failures	Uploaded content is re-read and hashed by the server rather than trusted from the client. Versions can be locked immutable in storage for a retention period. The audit log is hash-chained, so an altered or missing entry is detectable. Schema migrations are checked for data-destroying operations, and a destructive one must be listed with a reason before the build will pass.
A09 Security logging and monitoring failures	Every user-facing change is audited to an append-only, hash-chained log with a retention policy, write-once archival segments, export, and signed streaming to a SIEM. Requests carry a correlation identifier end to end, and every external seam can be traced in full detail when needed. Logs never contain a password, a token or a signed URL.

A10 Server-side request forgery	The two places that accept a caller-supplied URL — an organisation’s audit webhook and a client’s push end-point — are checked when the URL is registered and again at the moment of connecting , against the address actually resolved, connecting to that address so nothing can be re-pointed in between. Redirects are refused. Private ranges are refused unless the deployment’s own configuration permits them, and the cloud metadata addresses are refused regardless.
--	--

19.3 ASVS Level 2, in summary

Architecture	Layering is enforced by tests rather than convention; every architectural decision is recorded, including the ones later reversed.
Authentication	Standards-based, second factors supported and enforceable, throttled. Password composition rules are the one substantial item still open.
Session management	Server-issued tokens with bounded lifetimes; signing out clears both the server’s session and the client’s cached token.
Access control	Per-object, deny by default, and evaluated server-side on every request — the clients never decide it. They additionally hide an action the server did not advertise, so a user is rarely offered something that would be refused; but the hiding is courtesy, and the refusal is the control.
Validation & encoding	Typed data access, framework-escaped output, content checked on upload, and uploads that never pass through the application.
Logging	Structured, correlated, redaction-conscious; the audit trail is separate from the operational log and is tamper-evident.
Data protection	Content in object storage reached only by short-lived signed URLs; deletion is a two-stage process with a recycle bin, and disposal is auditable.
Communications	Transport security enforced outside development; the demonstration stack’s relaxations are development-only by construction.
Configuration	Dependency, licence, secret and image scanning on every build; a start-up gate that refuses development settings in production.

19.4 What is still open

An assessment that reports nothing outstanding is one to distrust. These are tracked, and none of them is a remotely exploitable defect in the shipped configuration:

- **Password composition** — no minimum length or breached-password check yet. Throttling bounds how fast a password can be guessed; it does nothing about one that is already on a public list.
- **Token lifetimes and refresh-token reuse detection** — lifetimes sit at framework defaults, and a replayed refresh token is not yet detected as a replay.
- **Two secrets at rest** — one-time-code seeds and external-link tokens are stored as issued unless a secrets manager is configured. Reading them requires database access.
- **Authentication events in the audit trail** — a failed sign-in is a log line, not yet an audit event.
- **Supply-chain attestation** — images are scanned but not signed, and no bill of materials is published.
- **Explicit XML hardening** — safe today by framework default rather than by assertion, which means nothing would notice if the default changed.
- **Proxy trust** — when the reverse-proxy header option is enabled it trusts any proxy, which is right for a local test network and too broad for production.

19.5 How this stays true

Every one of the claims above is held by something that runs on each change: a test, a gate, or both. The build fails on a dependency with a known vulnerability, on a licence outside the allowed set, on a leaked secret, on a destructive migration without a stated reason, and on a compiler warning. The rest is held by roughly two thousand automated tests, including end-to-end suites that drive the real application over HTTP and a real browser. That is the only reason a document like this one can be published rather than merely written.

20 Index

Terms are indexed where the manual explains them, not at every mention — the page listed is the one worth turning to.

ACL 3
Annotations 31
App password 42
Approval workflow 36
Audit trail 37
Auto-rotate 10
Auto-straighten (deskew) 10
Availability 20
Block 20
Booking 19
CalDAV 18, 27
Calendar 15
CardDAV 18, 27
Chat / comments 31
Check-out 6, 32
Click-to-copy 5
Contacts 15
Desktop client 3
Document management system 3
Edit profile 37
Executable content 7
External link 33
Facets 30
Follow / subscribe 31
Hit highlighting 5
IMAP 14, 27
Import / export 6
Index fields 3
Intray 6
Join items 9
Legal hold 36
Maintenance 20
Mask (document type) 3
Meeting room 19
Multi-factor authentication 4
My work 31
Name conflict 7
Notes 14
Notifications 31
OCR 29
Offered time 20
Open a document 5
Partial-word search 30
Personal repository 11
Preview 4
Recycle bin 36
Reference 14
Reference (shortcut) 11
Reminder 31
Repository 3
Retention 36
Rights 3
Rotate pages 9
Rotate/Sort 9, 10
Schedule 20
Search 30
Sensitivity label 11
Separator sheets 9
Server manager 37
Sign-in throttling 4
Sort pages 9
Split into pages 9
Storage quota 37
Tablet 21
Tags 11
Tasks 36
Tenant 37
Upload 6
Users & groups 37
Version 3
Version comment 8
Web client 3
WebDAV 12, 27
WebDAV deep link 12
Working copy 32
Zoom 5